

Table 4: Comparison of PREDIQL and baseline methods in terms of coverage performance. Tests that could not be executed are marked as FAILED.

API	ZAP	Burp	EvoMaster	GraphQLer	PREDIQL
UserWallet	50.00%	7.69%	61.54%	92.31%	96.15% (+3.84%)
Countries	33.33%	50.00%	50.00%	50.00%	100% (+50.00%)
Rick&Morty	33.33%	0.00%	66.67%	66.67%	100% (+33.33%)
GraphQLZero	93.75%	93.75%	71.88%	93.75%	100% (+6.25%)
EHRI	10.53%	0.00%	84.21%	94.74%	100% (+5.26%)
TCGDex	66.67%	33.33%	100%	100%	100% (+0.00%)

$$\text{Coverage} = \frac{\text{\#Unique Successful Responses}}{\text{\#Unique Nodes}}$$

where *coverage* is the fraction of schema nodes that return valid, error-free data rather than just an HTTP 200 status. This metric reflects how much of the GraphQL schema is actually exercised by successful queries, providing a more accurate view of the API’s reliability and robustness.

4.2 Schema Coverage (RQ1)

Table 4 compares the API coverage achieved by PREDIQL against four baseline tools: ZAP, BurpSuite, EvoMaster, and GraphQLer. Across all evaluated APIs, PREDIQL consistently attains the highest or near-highest coverage, demonstrating its ability to exercise a larger portion of the GraphQL schema through valid, data-returning queries. Traditional black-box scanners such as ZAP and BurpSuite achieve limited coverage because they lack awareness of GraphQL’s hierarchical structure and query dependencies. EvoMaster performs better by generating dynamic requests, but still falls short on complex schemas. GraphQLer improves coverage by incorporating schema context, yet PREDIQL surpasses it in nearly all cases by integrating retrieval-augmented reasoning and adaptive query generation. Note that the reported PREDIQL results correspond to its configuration with the best-performing LLM.

Table 5 presents the performance of tested large language models across different GraphQL APIs. Overall, all models achieve high coverage on simpler or well-structured APIs such as *Countries*, *Rick & Morty*, *GraphQLZero*, and *TCGDex*, indicating that these schemas are easier to navigate and query successfully. However, differences emerge on more complex or noisy schemas such as *UserWallet* and *EHRI*. GPT-5 Mini and Gemini 2.5 consistently produce the most stable and complete results, suggesting stronger schema understanding and query adaptation capabilities. LLaMA 3.1 performs comparably but occasionally misses certain paths. In contrast, DeepSeek R1 positions itself as the second-most proficient model on specific APIs, closely following GPT-5 Mini. These findings suggest that stronger reasoning models maintain high coverage under schema complexity, indicating better adaptability and understanding of GraphQL structures.

Conclusion for RQ1

PREDIQL consistently outperforms all baseline tools in schema coverage across diverse APIs. Its context-aware input inference and semantic reasoning enable more accurate and comprehensive query generation. On average, PREDIQL achieves a 16% improvement with a maximum improvement of 50% in schema coverage over the second-best model.

4.3 Prompt Engineering Impact (RQ2)

Prompt engineering is central to steering the reasoning capabilities of LLMs in improving schema coverage and discovering vulnerabilities in APIs. To address RQ2, we conduct an ablation study that isolates the contribution of each prompt enrichment component. This analysis quantifies how each module influences schema coverage and overall testing effectiveness. The corresponding configurations are detailed below.

- ◊ **PREDIQL-BASE**. The baseline configuration provides only the minimal schema context and expected response format. It guides the model to produce syntactically valid GraphQL queries.
- ◊ **PREDIQL-AQG**. Building on the baseline, the adaptive query generation configuration integrates both multi-armed bandit selection and retrieval-augmented generation, which is essential for *known values* ARM setting to enable adaptive and context-aware query synthesis.
- ◊ **PREDIQL-SCL**. The self-correction feedback loop configuration enhances the baseline prompt, adding an error-aware refinement cycle. Failed or invalid queries are logged with their corresponding error messages and reinjected into subsequent prompts as a corrective context.
- ◊ **PREDIQL**. The full pipeline, combining adaptive query generation, retrieval augmentation, and self-correction into a single closed-loop system.

Table 6 evaluates the contribution of individual prompt engineering components: the base prompt (BASE), the self-correction loop (PREDIQL-SCL), and adaptive query generation (PREDIQL-AQG). Contributions are computed for models that experienced enhancements from PrediQL-Base to PrediQL. Across models and APIs, the incremental addition of these modules consistently improves schema coverage, confirming that each contributes complementary benefits. PREDIQL-BASE alone often yields limited coverage, particularly in complex schemas such as *UserWallet* and *EHRI*, where naive prompting fails to satisfy nested or dependent field constraints. Introducing the self-correction loop (PREDIQL-SCL)

Table 5: Coverage achieved by different language models across GraphQL APIs.

API	LLaMA 3.1	Gemini 2.5	GPT-5 Mini	DeepSeek R1
UserWallet	88.46%	96.15%	96.15%	88.46%
Countries	100%	100%	100%	100%
Rick&Morty	100%	100%	100%	100%
GraphQLZero	100%	100%	100%	100%
EHRI	78.94%	100%	100%	100%
TCGDex	100%	83.33%	100%	100%

markedly reduces repeated schema violations, increasing coverage by 10–25% depending on the model capacity. Adaptive query generation (PREDIQL-AQG) provides an additional boost to APIs that require realistic parameter inference, increasing coverage by as much as 26% on EHRI and 15% on GraphQLZero for Gemini 2.5. When both mechanisms are combined in PREDIQL, coverage approaches or reaches 100% in almost all APIs and models.

Smaller open-source models such as LLaMA 3.1 follow the same pattern but exhibit slightly higher variance, reflecting reduced stability in long-context reasoning. Larger models (GPT-5 Mini, DeepSeek R1) demonstrate less sensitivity to prompt enrichment, suggesting that reasoning-oriented architectures benefit more from the feedback of retrieval and correction than from raw scale alone. In general, ablation confirms that the design gains of PREDIQLstem from the synergy of retrieval grounding, adaptive prompting, and iterative self-correction, rather than from model size alone.

Conclusion for RQ2

Overall, prompt engineering significantly enhances PREDIQL’s ability to achieve broader and more accurate schema coverage. By integrating adaptive query generation and self-correction, the system effectively adapts, learns, and refines its queries over iterations. This synergy results in a more intelligent, reliable, and efficient API exploration process.

4.4 Vulnerability Detection (RQ3)

Table 7 summarizes the vulnerability detection results across GraphQLer and the different configurations of PREDIQL. Among existing GraphQL testing tools, only GraphQLer includes a built-in vulnerability detection module, while others, such as EVO MASTER and ZAP, primarily focus on coverage measurement or payload

fuzzing. Therefore, GraphQLer serves as the most relevant baseline for assessing the detection capacity.

Across all evaluated APIs, PREDIQL consistently identifies a greater number and a wider range of vulnerabilities. While GraphQLer mainly exposes schema-level and input validation flaws, PREDIQL leverages retrieval-augmented reasoning and adaptive arm selection to detect deeper logic- and context-dependent weaknesses such as HTML injection, SSRF, and OS command injection. The variants PREDIQL -Gemini and PREDIQL -GPT-5 achieve the highest detection counts, improving unique findings by 20-40% on complex benchmarks such as UserWallet and GraphQLZero. In general, these results confirm that LLM-guided reasoning substantially enhances vulnerability discovery beyond static or heuristic testing baselines.

Qualitative analysis shows that PREDIQL detects a broader spectrum of vulnerability categories compared to GraphQLer, including deeper logic and context-dependent flaws. Its reasoning traces link each issue to its execution context (e.g., leaked variables or inconsistent authorization responses), enabling precise, evidence-based triaging.

Conclusion for RQ3

PREDIQL substantially advances vulnerability discovery beyond rule-based baselines. Its context-aware reasoning enables the LLM to correlate schema structure, response semantics, and execution traces, revealing logic and injection flaws that static or signature-driven tools overlook. Across all evaluated APIs, PREDIQL achieves broader and deeper vulnerability coverage, demonstrating that adaptive, retrieval-guided analysis is essential to uncovering complex security weaknesses in GraphQL APIs.

Table 6: Ablation study on prompt engineering components. BASE, AGQ, and SCL denote PREDIQL-BASE, PREDIQL-AGQ, and PREDIQL-SCL, respectively.

[illegible]

Table 7: Comparison of vulnerability detection performance between GraphQLer and PREDIQL variants.

API	GraphQLer		PREDIQL (LLaMA 3.1)		PREDIQL (Gemini 2.5)		PREDIQL (GPT-5 Mini)		PREDIQL (DeepSeek R1)	
	Vulnerabilities	Categories	Vulnerabilities	Categories	Vulnerabilities	Categories	Vulnerabilities	Categories	Vulnerabilities	Categories
UserWallet	26	7	31	11	41	7	20	6	34	8
Countries	6	2	7	3	9	2	9	4	7	3
Rick&Morty	12	3	10	10	13	4	11	4	14	6
GraphQLZero	37	8	35	7	37	7	44	6	34	7
EHRI	11	3	15	12	21	2	26	2	3	3
TCGDex	6	1	7	1	10	2	8	2	7	2

5 Discussion

Our evaluation demonstrates that PREDIQL consistently outperforms all existing GraphQL testing frameworks. This section discusses key observations, broader implications, and open research directions.

Impact of Model Size. Larger models such as GPT-5 Mini and DeepSeek R1 achieved higher semantic coherence and reasoning stability, while smaller open-source models (e.g., LLaMA 3.1) remained competitive at a fraction of the computational cost. This highlights a practical trade-off between reasoning depth and efficiency. When equipped with retrieval memory and adaptive prompting, lightweight models can approximate the performance of proprietary ones. A promising direction is to adopt hybrid configurations, using larger models for seed generation and schema understanding, followed by smaller models for iterative fuzzing, to balance throughput, coverage, and cost.

Implications Beyond GraphQL. The introduced mechanisms in PREDIQL, adaptive arm selection, self-corrective prompting, and retrieval-grounded reasoning, are not specific to GraphQL. These ideas extend naturally to other structured interface testing domains such as REST, gRPC, and JSON-RPC [38]. More broadly, PREDIQL demonstrates that retrieval-augmented reasoning and bandit-driven exploration can complement traditional coverage-guided and evolutionary fuzzing. Integrating symbolic reasoning or static program analysis into such adaptive loops may bridge the gap between semantic understanding and execution-level precision, enabling more generalizable automated security testing frameworks.

Limitations. While effective, PREDIQL is not without constraints:

- **Execution cost and rate limits.** LLM-guided fuzzing remains computationally intensive, and API rate throttling can slow feedback cycles.
- **Context window constraints.** Even with retrieval augmentation, large schemas can exceed model context limits, leading to partial prompt conditioning and missed relationships.
- **Response interpretation ambiguity.** The context-aware detector can identify likely vulnerabilities, but some cases still require human validation to confirm exploitability.
- **Model bias and non-determinism.** Variations in model architecture and decoding strategies lead to inconsistent results, motivating ensemble or calibration techniques for reproducibility.

Future Work. The growing ecosystem of agentic LLM frameworks offers a natural evolution path for PREDIQL. A multi-agent design,

with specialized agents for query generation, evaluation, and refinement, could enable continuous self-improvement and deeper exploit discovery. Another direction is the development of domain-specialized LLMs for API and schema reasoning, analogous to text-to-SQL models, which could reduce prompt overhead while improving precision and generalization. Finally, exploring hybrid systems that couple LLM reasoning with program analysis or formal verification could enable both semantic adaptability and provable assurance.

6 Conclusion

PREDIQL shows that the combination of retrieval, reasoning, and adaptive learning can fundamentally improve the way GraphQL APIs are tested. By integrating large language models into the fuzzing loop, it transforms random exploration into guided reasoning, allowing the system to understand schemas, infer dependencies, and generate purposeful queries. Through multi-armed bandit strategy selection, PREDIQL learns which testing behaviors yield the most valuable feedback, achieving higher coverage and uncovering vulnerabilities that existing tools consistently miss. Beyond its empirical gains, PREDIQL reveals a deeper insight: LLMs can act not only as generators but as analysts that interpret system behavior. This ability to connect input, responses, and context enables the detection of complex logic-level flaws that evade rule-based or pattern-driven scanners. These results mark a step towards autonomous, self-improving security testing, where models learn from every execution to test smarter over time. Future extensions will explore collaborative, multi-agent setups and large-scale retrieval across heterogeneous APIs, paving the way for intelligent systems that continuously learn the structure and weaknesses of modern Web applications.

References

- [1] Burp suite. <https://portswigger.net/burp>. Accessed: 2025-10-03.
- [2] Countries graphql api. <https://countries.trevorblades.com/>. Accessed: October 2025.
- [3] Deepseek r1 vs gpt-5 mini (model comparison). <https://aimodels.fyi/compare/deepseek-r1-vs-gpt-5-mini>. Accessed 2025-10-07.
- [4] European holocaust research infrastructure (ehri) graphql api. <https://portal.ehri-project.eu/api/graphql>. Accessed: October 2025.
- [5] Gemini 2.5 flash model. <https://deepmind.google/models/gemini/flash/>. Accessed 2025-10-07.
- [6] Gpt-5 mini vs gemini 2.5 flash (model comparison). <https://artificialanalysis.ai/models/comparisons/gpt-5-mini-vs-gemini-2-5-flash>. Accessed 2025-10-07.
- [7] GraphQL security (official guide). <https://graphql.org/learn/security/>. Accessed 2025-10-04.
- [8] GraphQL vs REST api — difference between api design styles. <https://aws.amazon.com/compare/the-difference-between-graphql-and-rest/>. Accessed 2025-10-04.
- [9] GraphQLzero api. <https://graphqlzero.almansi.me/>. Accessed: October 2025.
- [10] Meta llama 3.1 8b. <https://huggingface.co/meta-llama/Llama-3.1-8B>. Accessed 2025-10-07.
- [11] Rick and morty graphql api. <https://rickandmortyapi.com/graphql>. Accessed: October 2025.
- [12] Tcgdex graphql api. <https://api.tcgdex.net/v2/graphql>. Accessed: October 2025.
- [13] Userwallet graphql api. <https://github.com/graphql-compose/graphql-compose-examples/tree/master/examples/user-wallet>. Accessed: October 2025.
- [14] Owasp zed attack proxy (zap). <https://www.zaproxy.org/>. 2024.
- [15] AGRAWAL, S., AND GOYAL, N. Analysis of thompson sampling for the multi-armed bandit problem. *arXiv preprint arXiv:1111.1797* (2011).
- [16] ANDERSSON, T. Rest api vs graphql — a literature and experimental study. <https://www.diva-portal.org/smash/get/diva2:1571154/FULLTEXT01.pdf>, 2021.
- [17] ARCURI, A., GALEOTTI, J. P., MARCULESCU, B., AND ZHANG, M. Evomaster: A search-based system test generation tool. *Journal of Open Source Software* 6, 57 (2021), 2153.
- [18] BELHADI, A., ZHANG, M., AND ARCURI, A. Random testing and evolutionary testing for fuzzing graphql apis. *ACM Transactions on the Web* (2023).
- [19] BELHADI, Y., AND ARCURI, A. Evomaster for GraphQL: Black-box test generation for web APIs. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023), IEEE, pp. 1503–1507.
- [20] CAVENAGHI, E., ET AL. Non-stationary multi-armed bandit: Empirical evaluation of f-discounted-sliding-window thompson sampling. *Entropy* 23, 3 (2021), 380.
- [21] CHEN, C., DOLAN-GAVITT, B., AND LIN, Z. Elfuzz: Efficient input generation via llm-driven synthesis over fuzzer space. In *Proceedings of the 34th USENIX Security Symposium* (2025), USENIX Association. to appear.
- [22] DENG, Y., XIA, C. S., PENG, H., YANG, C., AND ZHANG, L. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. *arXiv preprint arXiv:2212.14834* (2022).
- [23] DOUZE, M., GUZHVA, A., DENG, C., JOHNSON, J., SZILVASY, G., MAZARÉ, P.-E., LOMELI, M., HOSSEINI, L., AND JÉGOU, H. The faiss library.
- [24] DOYENSEC. Crackql: GraphQL security testing tool. <https://github.com/doyensec/CrackQL>, 2021. Accessed: 2025-10-04.
- [25] ESCAPE. The state of graphql security 2024. Tech. rep., Escape Technologies, 2024. Accessed: October 2025.
- [26] ESCAPE TECHNOLOGIES. GraphQL-cop: Security scanner for graphql apis. <https://github.com/escape-technologies/graphql-cop>, 2023. Accessed: 2025-10-04.
- [27] FASTLY. Exploring the security implications of GraphQL. <https://www.fastly.com/blog/exploring-the-security-implications-of-graphql>, 2022.
- [28] FIORALDI, A., MAIER, D., EISSFELDT, H., AND HEUSE, M. Afl++: Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)* (2020).
- [29] FORWARD SECURITY. Autogql: Auto GraphQL scanner for burp suite. <https://github.com/FWDSEC/burp-auto-gql>, 2023. Accessed: 2025-10-07.
- [30] GOOGLE. Oss-fuzz – continuous fuzzing for open source software. <https://github.com/google/oss-fuzz>, 2024. Accessed: 2025-10-04.
- [31] GRAPHCRAWLER PROJECT. Graphcrawler: Automated graphql introspection and fuzzing tool. <https://github.com/gsmith257-cyber/GraphCrawler>, 2022. Accessed: 2025-10-04.
- [32] HATFIELD-DODDS, Z., ET AL. Deriving semantics-aware fuzzers from web API schemas, 2021.
- [33] HUANG, L., ZHAO, P., CHEN, H., AND MA, L. Large language models based fuzzing techniques: A survey. *arXiv preprint arXiv:2402.00350* (2024).
- [34] HYGRAPH. GraphQL survey 2024, 2024. Accessed: October 2025.
- [35] IBM PTC SECURITY. Denial of service attacks with GraphQL. https://medium.com/@ibm_ptc_security/denial-of-service-attacks-with-graphql-77189a6ba85b, 2023.
- [36] JIN, Y., ET AL. Fuzzgpt: Harnessing large language models for effective api fuzzing. In *Proceedings of the 33rd USENIX Security Symposium* (2024).
- [37] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (2019).
- [38] JSON-RPC WORKING GROUP. Json-rpc 2.0 specification. <https://www.jsonrpc.org/specification>, 2013. Origin date 2010-03-26, updated 2013-01-04, accessed 2025-10-07.
- [39] KESARWANI, M., GHOSH, S., GUPTA, N., CHAKRABORTY, S., SINDHGATTA, R., MEHTA, S., EBERHARDT, C., AND DEBRUNNER, D. GraphQL query generation: A large training and benchmarking dataset. pp. 1595–1607.
- [40] LATTIMORE, T., AND SZEPESVÁRI, C. *Bandit Algorithms*. Cambridge University Press, 2020.
- [41] LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., KARPUKHIN, V., GOYAL, N., KÜTTLER, H., LEWIS, M., YIH, W.-T., ROCKTÄSCHEL, T., ET AL. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* (2020), vol. 33, pp. 9459–9474.
- [42] LLVM PROJECT. libfuzzer – a library for coverage-guided fuzzing. <https://llvm.org/docs/LibFuzzer.html>, 2024. Accessed: 2025-10-04.
- [43] MADAAN, A., ZHOU, S., ALON, U., SONKAR, S., GUPTA, P., YANG, Y., AND NEUBIG, G. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)* (2023).
- [44] MCFADDEN, S., MAUGERI, M., HICKS, C., MAVROUDIS, V., AND PIERAZZI, F. Wendigo: Deep reinforcement learning for denial-of-service query discovery in graphql. In *2024 IEEE Security and Privacy Workshops (SPW)* (2024), pp. 68–75.
- [45] MENG, R., DUCK, G. J., AND ROYCHOUDHURY, A. Large language model assisted hybrid fuzzing. *arXiv* (2024).
- [46] OUYANG, L., WU, J., JIANG, X., ALMEIDA, D., WAINWRIGHT, C. L., MISHKIN, P., ZHANG, C., AGARWAL, S., SLAMA, K., RAY, A., SCHULMAN, J., HILTON, J., MILLER, L., SIMENS, M., ASKELL, A., WELINDER, P., CHRISTIANO, P., LEIKE, J., AND LOWE, R. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155* (2022).
- [47] PERERA, I., ABEYRATHNE, H., MALALGODA, S., AND IFTHIKAR, A. Enhancing graphql security by detecting malicious queries using large language models, sentence transformers, and convolutional neural networks. *arXiv preprint arXiv:2508.11711* (2025). arXiv:2508.11711v1 [cs.CR].
- [48] REIMERS, N., AND GUREVYCH, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP* (2019), pp. 3982–3992.
- [49] TECHTARGET. What’s next for apis? 4 api trends for 2025 and beyond, 2024. Accessed: October 2025.
- [50] TSAI, Y., ZHANG, T., ET AL. GraphQLer: Enhancing graphql security with context-aware api testing. *arXiv preprint arXiv:2504.13358* (2025).
- [51] XIA, C., WANG, R., MENG, Y., TANG, Z., WANG, Y., XUE, Y., LI, X., AND CHEN, X. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)* (2024), ACM, pp. 1–13.
- [52] ZHANG, T., ET AL. Dfuzz: Large language models for deep api fuzzing with white-box guidance. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)* (2025), IEEE/ACM.